

AI documentatie

1. Reflectie

Origineel hield ik al mijn AI interacties bij in een markdown bestand in mijn repository. Dit komt omdat in mijn IDE de AI assistent geïntegreerd is en het zo makkelijker was om alles snel bij te houden op één plek.

Als IDE gebruik ik Cursor. Dit is een fork van vscode en ziet er ongeveer hetzelfde uit, maar de AI assistent is net iets beter geïntegreerd.

Ik vind het enorm handig om code te selecteren en als context toe te voegen in de chat met de AI assistent. Ik gebruik dit vaak om uitleg te vragen over dingen die ik niet begrijp, voor debugging bij errors en om suggesties te vragen tijdens het refactoren.

De autocomplete werkt ook vaak heel goed. Als ik eerst een duidelijke comment schrijf van wat het stuk code moet doen, geeft het vaak een correcte suggestie wanneer ik aan het typen ben. Het kan ook wel enorm storend zijn wanneer de suggesties niet kloppen waardoor ik soms de autocomplete gewoon uitschakel.

Ik merk wel dat de chat met de assistent mijn eerste lijn voor hulp is geworden in plaats van dingen op te zoeken in documentatie, stackoverflow, ...

Enerzijds is het gewoon enorm gemakkelijk om dat snel te kunnen doen in een tab die rechtstreeks in je IDE zit. Het bespaart heel wat tijd en voor simpele zaken werkt dat heel goed.

Anderzijds ben ik er ook wel van bewust dat dit heel gevaarlijk kan zijn en je niet blind mag volgen wat er gezegd wordt. Als ik iets niet begrijp, vraag ik door en als de uitleg mij niet kan overtuigen, ga ik online verder zoeken naar een duidelijk antwoord.

Ik ben er wel van overtuigd dat de voordelen nu al doorwegen op de nadelen. Indien je zelf met een gezonde dosis scepticisme de antwoorden kan interpreteren, haal je de fouten er wel makkelijk uit. Je kan dan meteen doorvragen en uiteindelijk nog altijd veel sneller tot een juist antwoord komen dan wanneer je door documentatie of fora moet bladeren.

Ik gebruik AI dan ook echt als een tool om te leren, niet om oplossingen voor te kauwen die ik zonder nadenken kopieer in mijn code.

Zelfs in de theorie delen van de cursus vraag ik regelmatig wat verduidelijking of extra voorbeelden tot ik het gevoel heb dat ik een concept goed begrijp. Als ik via google dezelfde informatie moet zoeken, ben ik snel heel veel tijd kwijt.

2. Chat logs

2.0 Project

Ik had zelf wel wat ideeën voor nieuwe features maar heb ook eens gevraagd naar wat extra inspiratie

I'm working on a machine learning project where i need to predict the ratings for tv shows. I have some open data for tv shows with these features: name, channel, date, starting hour and duration.

I'm going to add open weather data and the times of sunset and sunrise as extra data. Can you think of some interesting features that would be an added value?

I'll help you brainstorm some interesting features that could potentially improve your TV show ratings prediction model. Here are several feature ideas that could add value:

1. Time-based Features:
 - Day of the week (weekday vs weekend)
 - Month/Season (to capture seasonal viewing patterns)
 - Holiday indicators (national holidays, special events)
 - Time slot category (prime time, late night, morning)
 - Competition level (number of other shows airing in the same time slot)
2. Content-related Features:
 - Genre of the show
 - Whether it's a new episode or rerun
 - Episode number in the season
 - Whether it's a season premiere or finale
 - Show's historical average rating
 - Show's age (how long it's been on air)
3. Weather-related Features (extending your weather data):
 - Temperature
 - Precipitation probability
 - Weather conditions (sunny, rainy, etc.)
 - Wind speed
 - Humidity
 - Air quality index
 - UV index
4. Time of Day Features (extending your sunrise/sunset data):
 - Hours of daylight
 - Whether the show airs during daylight or darkness
 - Golden hour indicator (hour before sunset)
 - Blue hour indicator (hour after sunset)
5. Social/Event Features:

- Major sports events happening at the same time
 - Competing major TV events (awards shows, special broadcasts)
 - Social media trends/hashtags related to the show
 - News events that might affect viewership
6. Channel-specific Features:
- Channel's average viewership
 - Channel's target demographic
 - Whether the show is on a premium channel
 - Channel's market share
 - Channel's programming strategy (e.g., if they typically air similar shows)
7. Historical Performance Features:
- Rolling average of ratings for the same time slot
 - Rolling average of ratings for the same show
 - Previous episode's rating
 - Season-to-date average rating
 - Year-over-year comparison for similar time slots
8. Audience-related Features:
- Target demographic information
 - Historical audience retention rates
 - Lead-in show's rating (if applicable)
 - Lead-out show's rating (if applicable)
9. Technical Features:
- Whether the show is available on streaming platforms
 - Whether it's available in HD/4K
 - Whether it's available with closed captions
 - Whether it's available in multiple languages
10. Competitive Features:
- Number of competing shows in the same time slot
 - Genre of competing shows
 - Channel strength of competing shows
 - Whether major competing shows are new episodes or reruns

These features could be particularly valuable because:

- They capture different aspects of viewer behavior
- They account for external factors that might influence viewing decisions
- They provide historical context for the show's performance
- They consider the competitive landscape
- They account for environmental factors (weather, time of day)
- To implement these features, you would need to:
- Gather the necessary data from various sources
- Preprocess the data to ensure it's in a format suitable for your model
- Handle missing values appropriately
- Consider feature engineering to create more complex features

- Use feature selection techniques to identify the most impactful features

2.1 Data verzamelen

How can I get end_date as the day before yesterday? It should be in the same format as start_date where months and days shouldn't have leading zeros.

I'll help you get the end_date in the same format as start_date. Here's how you can do it using Python's datetime module:

```
from datetime import datetime, timedelta

# Get the day before yesterday
end_date = (datetime.now() -
            timedelta(days=2)).strftime("%Y-%m-%d")
```

However, I should note that the %m and %d format codes (which remove leading zeros) only work on Unix-like systems (Linux, macOS).

...

How can i add df_current_day to the df I initialised at the top?

To add df_current_day to your main DataFrame df, you can use pandas' concat function.

```
df = pd.concat([df, df_current_day], ignore_index=True)
```

This will:

1. Use pd.concat() to combine the existing df with the new df_current_day
2. ignore_index=True will reset the index of the resulting DataFrame to be sequential (0, 1, 2, ...)

Error fetching data: ('Connection aborted.', RemoteDisconnected('Remote end closed connection without response'))

Would this point to a rate limiting issue?

Yes, this error message suggests a rate limiting issue. The "Remote end closed connection without response" typically occurs when the server actively closes the connection, which is a common way for APIs to implement rate limiting.

Looking at your code, you're making requests for each day in sequence without any delay between requests. This could trigger the API's rate limiting protection. Here's how you can modify your code to handle this:

...

```

while current_date <= end:
    try:
        # Format date back to string
        date_str = current_date.strftime("%Y-%m-%d")
        print(date_str)

        # Get data
        df_current_day = get_cim_tv_data(date_str)

        # Add to df
        if not df_current_day.empty:
            df = pd.concat([df, df_current_day],
ignore_index=True)
        else:
            print(f"No data for date: {date_str}")

        # Add a delay between requests to avoid rate limiting
        time.sleep(1)

        # Move to next day
        current_date += timedelta(days=1)

    except Exception as e:
        print(f"Error processing date {date_str}: {str(e)}")
        time.sleep(5) # Wait 5 seconds after an error
        continue

```

You might need to adjust the sleep times (1 second and 5 seconds) depending on the API's rate limits. If you're still getting rate limit errors, you could:

1. Increase the delay between requests
2. Implement exponential backoff (increase wait time after each error)
3. Check if the API has a documented rate limit and adjust accordingly

Hier had ik de code als context toegevoegd in het chatvenster en gewoon de error gekopieerd als prompt.

invalid literal for int() with base 10: '971420,70'

Ah, I see the issue. When you try to convert the string to an integer, it's failing because of the comma. We need to handle the decimal numbers properly. Here's how we can fix it:

```

# First remove dots and replace commas with dots for proper
decimal handling
ratings_df['Kijkers'] = ratings_df['Kijkers'].str.replace('.',
').str.replace(',', '.')

```

```
# Convert to float first (to handle decimal numbers)
ratings_df['Kijkers'] = pd.to_numeric(ratings_df['Kijkers'],
errors='coerce')

# Then convert to integer (this will round the decimal numbers)
ratings_df['Kijkers'] = ratings_df['Kijkers'].astype(int)
```

This will:

1. Remove the dots (thousand separators)
2. Replace commas with dots (to handle decimal numbers properly)
3. Convert to float first (which can handle decimal numbers)
4. Finally convert to integer (which will round the decimal numbers)

So numbers like:

- "971.420,70" will become 971420.70 (float) and then 971421 (integer)
- "836.931,30" will become 836931.30 (float) and then 836931 (integer)

This should handle all the numeric values correctly, including those with decimal places. The `errors='coerce'` parameter will convert any non-numeric values to NaN, which you can then handle separately if needed.

How can I get the number of the week in the year? Similar to the `day_of_week` property above

Based on the code you've shared, you're already using the correct method to get the week number of the year! The line:

```
ratings_df["Week"] = ratings_df["Datum"].dt.weekofyear
```

is exactly what you need. This method `dt.weekofyear` returns the week number (1-53) for each date in your datetime column.

However, I should note that `weekofyear` is actually deprecated in newer versions of pandas. The recommended way to get the week number is to use ``dt.isocalendar().week``. So if you want to update your code to use the more modern approach, you could write:

```
ratings_df["Week"] = ratings_df["Datum"].dt.isocalendar().week
```

Both methods will give you the same result - a number between 1 and 53 representing which week of the year the date falls in. The ISO calendar week numbering system is used, where:

- Weeks start on Monday
- Week 1 is the first week of the year that contains a Thursday
- Week numbers range from 1 to 53

Would you like me to explain more about how the week numbering works or help you with anything else?

I want to get historic and future sunrise and sunset data to add to this project. Do you know any open data platforms that provide this? Or other ways of getting this data?

Yes, there are several ways to get sunrise and sunset data. Here are some of the best options:

Astral Library (Recommended for Python)

- A Python library that calculates sunrise/sunset times based on location
- Very accurate and easy to use
- Can calculate for any date (past or future)
- No API key needed

Sunrise-Sunset API

- Free API with rate limits
- Requires API key for higher usage
- Historical and future data
- Example endpoint:
<https://api.sunrise-sunset.org/json?lat=50.8503&lng=4.3517&date=2024-03-20>

Open-Meteo API

- Free, no API key required
- Provides sunrise/sunset data along with other weather data
- Historical data available
- Example endpoint:
<https://api.open-meteo.com/v1/forecast?latitude=50.8503&longitude=4.3517&daily=sunrise,sunset>

NOAA Solar Calculator

- Very accurate calculations
- Can be used programmatically
- No API key needed
- More complex to implement but very reliable

For your project, I would recommend using the astral library because:

- It's free and open source
- No API key or rate limits
- Can calculate for any date (past or future)
- Very accurate
- Easy to integrate with pandas
- Works offline

2.2 Data ontdekken en visualiseren

I plotted histograms for these columns but i get some strange results for the weeks.

Do you have an explanation for this?

I calculated the weeks value like this: `data["Week"] = data["Datum"].dt.isocalendar().week`

Great question! Your code for calculating the week number is correct.

However, the histogram for "Week" shows some strange spikes, especially at the beginning and around week 27. Here are some possible explanations:

1. Partial Weeks at the Start and End of the Year
 - a. The ISO week numbering system means that the first and last week of the year may not have a full 7 days.
 - b. If your data starts or ends mid-week, those weeks will have fewer entries, but if you have multiple years, the first and last week of each year can accumulate more or fewer entries, causing spikes.
2. Data Collection Gaps or Anomalies
 - a. If there are missing dates or gaps in your data, some weeks may have more or fewer entries.
 - b. If your data source sometimes skips days (e.g., holidays, technical issues), this can create uneven week distributions.
3. Special Events or Outliers
 - a. Certain weeks (like holidays, New Year, or special events) may have more broadcasts, causing spikes.
 - b. For example, week 1 (New Year) and week 27 (mid-year, possibly summer holidays or special events) might have more or fewer programs.
4. Incorrect Handling of Week Numbers
 - a. If you have data from multiple years, but you only plot the week number (1–53) without considering the year, all years' week 1s are combined, all week 2s are combined, etc.
 - b. This can create artificial spikes if some weeks are overrepresented across years.
5. Data Duplication or Aggregation
 - a. If your data is aggregated or duplicated for certain weeks, this can also cause spikes.

I did find some duplicates but the issue still remains. What would you suggest to investigate further?

How to Investigate Further

Check the distribution of dates per week and year:


```
print(data.groupby(['Year', 'Week']).size())
```

Plot a heatmap of counts per week and year:

```
import seaborn as sns
week_year_counts = data.groupby(['Jaar',
'Week']).size().unstack(fill_value=0)
plt.figure(figsize=(15, 6))
sns.heatmap(week_year_counts, cmap="Blues")
plt.title("Number of entries per week and year")
plt.show()
```

Could it be because I'm dividing the data in 50 bins but we have more weeks?

Yes, that's a very good observation! The number of bins in your histogram can significantly affect how the data is visualized, especially for discrete variables like week numbers.

- There are up to 53 weeks in a year (ISO weeks).
- If you use bins=50, some bins will contain more than one week, and some weeks may be grouped together or split across bins.
- This can create artificial spikes or dips in your histogram, making the distribution look strange or misleading.

For discrete data like weeks, it's best to set the number of bins equal to the number of unique values:

```
data["Week"].hist(bins=data["Week"].nunique())
plt.title("Week Distribution")
plt.xlabel("Week")
plt.ylabel("Count")
plt.show()
```

2.3 Modellen testen

I have this regressorlist that i will run with (mostly) default parameters to see which one performs best and which one i can further finetune. Do you have any suggestions for other models that i can add?

1. Gradient Boosting Regressor (GBR)
 - a. Often performs very well on structured data
 - b. Good balance between performance and interpretability
 - c. Can handle non-linear relationships
2. XGBoost Regressor
 - a. One of the most powerful gradient boosting implementations
 - b. Excellent performance on many regression tasks
 - c. Has built-in regularization
3. LightGBM Regressor
 - a. Another efficient gradient boosting framework
 - b. Often faster than XGBoost
 - c. Good for large datasets
4. Elastic Net
 - a. Combines L1 and L2 regularization
 - b. Good for datasets with many features
 - c. Can handle multicollinearity
5. Kernel Ridge Regression
 - a. Good for non-linear relationships
 - b. More stable than SVR in some cases
 - c. Less sensitive to hyperparameters

Here's how you could add these to your regressor list:

...

A few notes:

1. You'll need to install additional packages for XGBoost and LightGBM:

...

2. For the imports, you'll need to add:

...

3. These models are added with their default parameters, but they all have extensive hyperparameter tuning possibilities if you find they perform well in your initial comparison.
4. If you're working with a large dataset, you might want to start with a subset of these models first, as some (like XGBoost and LightGBM) can be computationally intensive.

If I find that a stacking regressor works best. Do I need to run a gridsearch for each model to find the best parameters? And what for the final estimator? I can't really emulate the data it will receive?

This is a great question about hyperparameter tuning for StackingRegressor! Let me explain the best approach:

1. For Base Models:
 - a. Yes, you should perform GridSearchCV for each base model individually first
 - b. This is because the base models need to be well-tuned to provide good predictions for the meta-learner
 - c. You can use cross-validation to find the best parameters for each model separately
2. For the Final Estimator (Meta-learner):
 - a. You don't need to emulate the data it will receive
 - b. The meta-learner will receive the cross-validated predictions from the base models
 - c. You can still perform GridSearchCV on the final estimator, but you'll need to do it as part of the StackingRegressor

Here's an example of how you could implement this:

...

Some important points to note:

1. Cross-validation in StackingRegressor:
 - a. The cv parameter in StackingRegressor ensures that the meta-learner doesn't see the same data that was used to train the base models
 - b. This prevents data leakage and overfitting
2. Final Estimator Choices:
 - a. Common choices for the final estimator include:
 - i. LinearRegression (simple, interpretable)
 - ii. Ridge or Lasso (if you want regularization)
 - iii. ElasticNet (combines L1 and L2 regularization)
 - iv. LightGBM or XGBoost (if you want more complex relationships)
3. Computational Considerations:
 - a. This approach can be computationally expensive
 - b. You might want to start with a smaller parameter grid
 - c. Consider using RandomizedSearchCV instead of GridSearchCV for faster results
4. Best Practices:
 - a. Use the same cross-validation strategy throughout
 - b. Make sure to use the same scoring metric consistently

- c. Consider using a validation set to evaluate the final model
-

Can you think of any other parameters i could set to improve this model?

Yes, there are several important Random Forest parameters that could help improve the model's performance. Here's an expanded parameter grid with additional parameters and their explanations:

Key additions and explanations:

1. `min_samples_leaf`:
 - a. Controls the minimum number of samples required to be at a leaf node
 - b. Higher values can help prevent overfitting
 - c. Useful when you have noisy data
2. `max_features`:
 - a. Controls how many features are considered for the best split
 - b. 'sqrt' and 'log2' are common choices that can help prevent overfitting
 - c. None means all features are considered
3. `max_samples`:
 - a. Only applicable when `bootstrap=True`
 - b. Controls the size of the bootstrap samples
 - c. Can help with overfitting by using smaller subsets
4. `min_weight_fraction_leaf`:
 - a. Similar to `min_samples_leaf` but works with sample weights
 - b. Useful when you want to give more importance to certain samples
5. `criterion`:
 - a. Different splitting criteria can work better for different types of data
 - b. 'squared_error' is default and good for most cases
 - c. 'absolute_error' is more robust to outliers
 - d. 'poisson' is good for count data

To make the search more efficient, you could also add early stopping to prevent overfitting:

...

How does the oob_score help?

The Out-of-Bag (OOB) score is a very useful feature of Random Forest that helps in several ways:

1. Built-in Cross-Validation:
 - a. When Random Forest creates trees using bootstrap sampling, about 37% of the data is not used for training each tree (this is the "out-of-bag" data)
 - b. This OOB data can be used to evaluate the model without needing a separate validation set

- c. It's like having built-in cross-validation without the computational cost
2. Overfitting Detection:
- a. You can compare the OOB score with the training score
 - b. If there's a large gap between them, it indicates overfitting
 - c. This helps you tune parameters like `max_depth` and `min_samples_leaf`
-

How can I get the feature names from a grid search model?

To get the feature names from a GridSearchCV model, you'll need to access the best estimator and then get the feature names from the column transformer. Here's how you can do it:

```
# Assuming you have a GridSearchCV object named 'grid_search'
best_model = grid_search.best_estimator_

# If your model has a column transformer as part of the pipeline
column_transformer = best_model.named_steps['columntransformer']
# or whatever name you gave to your column transformer step
feature_names = column_transformer.get_feature_names_out()

print("Feature names:", feature_names)
```

If you're using a pipeline, the exact name of the column transformer step might be different. You can check the available steps in your pipeline using:

```
print(best_model.named_steps.keys())
```

This will show you all the named steps in your pipeline, and you can use the correct name to access the column transformer.

Would you like me to help you find the exact name of your column transformer step or explain any part of this in more detail?

I tried to improve the best model I found in a grid search by boosting it. But when I compare the results my boosted model has a MAPE of 32.6% and my original model has a score of 16.9%

Is that big of a difference normal? Does it point to overfitting on the boosted model?

Let me help analyze this situation. The significant difference in MAPE scores (32.6% vs 16.9%) is indeed concerning and suggests there might be issues with the boosted model. Let me explain why this might be happening:

1. Overfitting Concerns:
 - a. The large increase in MAPE (almost doubling) strongly suggests overfitting
 - b. AdaBoost can be particularly prone to overfitting when:
 - i. The base model (RandomForest in this case) is already complex
 - ii. The number of estimators is too high
 - iii. The data is noisy
2. Potential Issues in Your Implementation:
 - a. You're using a RandomForest with 600 estimators as the base model, which is already quite complex
 - b. Then you're boosting it with 100 additional estimators
 - c. This creates a very complex model that might be learning noise in the training data
3. Recommendations:
 - a. Try reducing the complexity of your base model first
 - b. Consider using a simpler base model like a DecisionTree with limited depth
 - c. Reduce the number of boosting iterations (n_estimators in AdaBoost)
 - d. You might want to try cross-validation to better understand the model's performance

Here's a suggested modification to your code:

...

The fact that your original RandomForest model performed better suggests that the additional complexity from boosting wasn't beneficial in this case. This is not uncommon - sometimes simpler models can outperform more complex ones, especially when the underlying patterns in the data are relatively straightforward.